ResolveFlow Concurrency Pass: Explained

What we changed in ResolveFlow, why, how each piece works, and how to talk about it in an interview. Branch: `feat/asyncio-contextvars-concurrency`. Code: `graph/nodes/fetch_evidence.py`, `graph/build.py`, `graph/request_context.py`, `app/limits.py`, `app/main.py`, `graph/nodes/execute.py`. Tests: `tests/test_concurrency.py`.

Contents

ResolveFlow Concurrency Pass: Explained
- Contents
- How to use this document
- Chapter 1. The whole change in one page
- Chapter 2. asyncio in fetch_evidence
- Chapter 3. ContextVar: request identity
- Chapter 4. Semaphore: bounding /api/analyze
- Chapter 5. The GIL: what we did and did not do
- Chapter 6. Tests: what each one proves
- Chapter 7. Honesty table: what you can and cannot claim
- Chapter 8. Talk track and drill
- Chapter 9. try / except / finally in async code
- Appendix. Files changed

How to use this document

1. Read **Chapter 1** so you can state the whole change in 60 seconds.
2. Read each of Chapters 2–5 as "problem → mechanism → code → why this choice".
3. Chapter 5 covers the GIL, Chapter 6 shows what each test proves, and Chapter 7 is the **honesty table**: what you may claim and what you may not.
4. Chapter 8 is the **talk track and a drill**. Chapter 9 covers `try/except/finally` with cancellation, the most likely follow-up probe. Close the doc and answer the drills.

Chapter 1. The whole change in one page

The situation before

ResolveFlow's API (`app/main.py` , FastAPI) runs a LangGraph pipeline per request: `fetch_evidence → normalize → classify → (diagnose → review) → await_approval → execute` .

Three problems, none of them bugs, all of them things a senior reviewer notices:

1. **Slow fetch.** `fetch_evidence` made five blocking GitHub calls one after another, even though most do not depend on each other.
2. **No request identity in logs.** Nothing tied a log line to a request or a user. In a multi-user service you cannot debug or audit a GitHub write ("which user caused this comment?") without that.
3. **No overload protection.** Any number of `/api/analyze` calls could run at once, each making paid LLM calls and holding worker threads. A burst would hit provider rate limits and starve cheap endpoints such as `/api/health` .

What we did

Problem	Tool	Where
Sequential GitHub calls	**asyncio**: `gather` + `to_thread`	`afetch_evidence` in `graph/nodes/fetch_evidence.py`
No request identity	**ContextVar**	`graph/request_context.py` , middleware in `app/main.py`
Unbounded concurrency	**Semaphore**	`AnalysisGate` in `app/limits.py` , used by `/api/analyze`
"What about the GIL?"	**Reasoning, not code**	`docs/CONCURRENCY.md` ; the choice of threads for I/O

Measured effect (fake 150 ms calls, not real GitHub)

```
sync  fetch_evidence   0.77 s   (5 calls in sequence)
async afetch_evidence  0.31 s   (longest chain: 2 calls)
```

That is a **simulated** number: each GitHub call is replaced by a function that sleeps 150 ms. It demonstrates the *shape* (sum of calls vs longest chain). It is not a latency measurement of the live system. See Chapter 7.

The 60-second version you can say out loud

> "The fetch step made five GitHub calls sequentially. Three of them are independent, and the other two are a dependent pair, so I run those as concurrent groups with `asyncio.gather` over `to_thread`. I kept the `requests` client because it is shared and security-relevant, and threads blocked on a socket release the GIL, so they genuinely overlap. For observability I put a trace id and user id in ContextVars so every log line, including the GitHub write audit line, is attributable to a request, and concurrent requests can't see each other's values. And I capped concurrent analyses per process with a semaphore that sheds load with a 503 after a bounded wait, rather than queueing forever. I tested the behaviours, and I'd measure the real before/after on live issues before quoting a number."

Chapter 2. asyncio in `fetch_evidence`

2.1 The problem, concretely

Original `fetch_evidence` (still there, kept as the reference path):

```
linked_pr  = github_client.get_linked_pull_request(...)   # 1
check_runs = github_client.get_check_runs(...) if linked_pr else []   # 2 (needs 1)
issue      = github_client.get_issue(...)                 # 3
comments   = github_client.get_comments(...)              # 4
codeowners = github_client.get_codeowners(...)            # 5  (itself tries up to 3 paths)
```

Each call is a network round trip of, say, 100–300 ms in which the CPU does nothing. Total latency is the **sum**.

2.2 The dependency graph (this is the real design work)

```
get_issue ──────────────────────────┐
get_comments ───────────────────────┤
get_codeowners ─────────────────────┼──> raw_evidence
get_linked_pull_request -> get_check_runs ┘
         (check_runs needs the PR's head SHA)
```

Only **one** real dependency exists: check-runs need the SHA from the linked PR. Everything else is independent. So the minimum possible latency is the **longest chain** (two calls), not the sum (five).

Getting this graph right is the point. Naively wrapping all five in `gather` would be wrong: `get_check_runs` needs `linked_pr["head"]["sha"]` before it can even be called.

2.3 The code

```
async def afetch_evidence(state: GraphState) -> dict:
    repo, issue_number = _parse_issue_url(state["issue_url"])
    token = state.get("github_token")

    async def pr_then_checks() -> list:
        linked_pr = await asyncio.to_thread(
            github_client.get_linked_pull_request, repo, issue_number, token=token)
        if not linked_pr:
            return []
        return await asyncio.to_thread(
            github_client.get_check_runs, repo, linked_pr["head"]["sha"], token=token)

    issue, comments, codeowners, check_runs = await asyncio.gather(
        asyncio.to_thread(github_client.get_issue, repo, issue_number, token=token),
        asyncio.to_thread(github_client.get_comments, repo, issue_number, token=token),
        asyncio.to_thread(github_client.get_codeowners, repo, token=token),
        pr_then_checks(),
    )
    return {"raw_evidence": {...}}
```

Reading it:

- `pr_then_checks()` is a small coroutine that keeps the dependent pair in order (await the PR, then await the checks).

- `gather(...)` starts all four things at once and waits for all of them. Three are single calls, one is the two-call chain.
- Results come back **in the order given**, regardless of which finishes first, which is why the tuple unpacking is safe.

2.4 `asyncio.to_thread`: how a blocking call becomes awaitable

`requests.get(...)` blocks. If you called it directly inside an `async def`, it would freeze the event loop (Q3 in the Python deep dive). `asyncio.to_thread(fn, *args, **kwargs)`:

1. Runs `fn` on a worker thread from the loop's default thread pool.
2. Returns an awaitable, so the event loop is free to run other coroutines meanwhile.
3. **Copies the current `contextvars` context** into the worker (this matters in Chapter 3).

2.5 Why threads give real overlap despite the GIL

The GIL lets only one thread run Python bytecode at a time. But a thread waiting in `socket.recv` (inside `requests`) has **released** the GIL. While four threads are all waiting on the network, none of them hold it, so all four requests are in flight simultaneously. The GIL would only be a bottleneck if the threads were doing heavy pure-Python computation, which they are not.

> Interview framing: "The GIL serialises bytecode, not waiting. This code spends its time waiting."

2.6 Decisions and the reasoning behind each

Decision	Alternative	Why we chose this
`to_thread` around `requests`	Rewrite `github_client` with `httpx.AsyncClient`	`github_client` is a deliberately thin client shared with `ingest.py` and the sync path, and it is the module that performs the GitHub writes. Rewriting a safety-relevant module for no gain on this workload is poor risk/benefit.
`asyncio.gather`	`asyncio.TaskGroup`	A thread already inside a blocking `requests` call **cannot be cancelled**, so TaskGroup's "cancel the siblings on first failure" would not actually stop them. TaskGroup also wraps failures in an `ExceptionGroup`, which would degrade the message `app/main.py` returns in its 502. `gather` re-raises the original `requests.HTTPError`. Calls are read-only, so letting stragglers finish is harmless.
Keep sync `fetch_evidence`	Delete it	`graph.invoke` (REPL/scripts) and the eval harness use the sync path.
Leave other nodes sync	Make LLM nodes async	Under `ainvoke`, LangGraph runs sync nodes in a thread pool automatically. Each of those nodes makes one LLM or retrieval call, so there is nothing to overlap. Also `eval/harness.py` calls `independent_review` synchronously; making it async would break the safety-gate regression suite.

2.7 Registering both variants in the graph

LangGraph needs to know there is an async version. In `graph/build.py`:

```
from langchain_core.runnables import RunnableLambda

graph.add_node("fetch_evidence", RunnableLambda(fetch_evidence, afunc=afetch_evidence))
```

`RunnableLambda(func, afunc=...)` holds two implementations of the same step. `graph.invoke(...)` calls `func`; `graph.ainvoke(...)` calls `afunc`. `app/main.py` uses `ainvoke`, so production takes the concurrent path, while sync callers are unaffected.

2.8 Error behaviour

If `get_issue` raises `requests.HTTPError` (for example 404), `gather` propagates that exact exception, same type as before. `app/main.py` catches it and returns 502 with the original message. A test (`test_async_fetch_propagates_original_exception_type`) pins this.

2.9 Follow-ups to be ready for

- "*Why not just threads (`ThreadPoolExecutor`)?*" It would work. `to_thread` + `gather` expresses the same thing inside the event loop that FastAPI already runs, and composes with the dependent chain naturally.
- "*What if GitHub rate-limits you with five parallel calls?*" Per request that is only four concurrent calls, and the semaphore (Chapter 4) bounds the number of requests, so total GitHub concurrency is bounded at about `4 × limit` per process. Real hardening would also honour `Retry-After` on 403/429 responses, which this code does not do.
- "*Does this change correctness?*" The result dictionary is identical to the sequential version. A test asserts exact equality.

Chapter 3. ContextVar: request identity

3.1 The problem

The service handles many users at once. When something goes wrong (or when a GitHub comment is posted), you need to know *which request and which user*. Passing `trace_id` and `user_id` through every function argument pollutes every signature. A module-level global would be shared by all concurrent requests, so user B's log line could carry user A's id. That is a correctness and privacy bug, not a style issue.

3.2 What a ContextVar is

A `ContextVar` holds a value that is **scoped to the current execution context**:

- Each asyncio **task** gets its own copy of the context. Two requests running concurrently in the same thread see different values.
- Setting a value returns a **token**; `var.reset(token)` restores the previous value exactly.
- When a new task is created, or `to_thread` is called, the context is **copied** into it.

It is the correct primitive for "per-request state in an async program", which is exactly what thread-locals were for in a threaded program.

3.3 The code (`graph/request_context.py`)

```
trace_id_var: ContextVar[str] = ContextVar("trace_id", default="-")
user_id_var:  ContextVar[str] = ContextVar("user_id",  default="-")

@contextmanager
def bind(*, trace_id=None, user_id=None):
    tokens = []
    if trace_id is not None:
        tokens.append((trace_id_var, trace_id_var.set(trace_id)))
    if user_id is not None:
        tokens.append((user_id_var, user_id_var.set(str(user_id))))
    try:
        yield
    finally:
        for var, token in reversed(tokens):
            var.reset(token)          # always restore, even if the block raised
```

Two details worth being able to explain:

- **`token` / `reset`, not `set(old_value)`.** Reset restores the exact previous state, including "was never set", and nests correctly.
- **`default="-"`.** Logging never crashes if a value was not set (health checks, startup).

3.4 Where the values get set (`app/main.py`)

1. **Middleware** assigns a `trace_id` for every request (reusing an inbound `X-Request-ID` if present) and returns it as a response header:

```
@app.middleware("http")
async def trace_requests(request, call_next):
    trace_id = (request.headers.get("x-request-id") or "").strip()[:64] or uuid.uuid4().hex[:16]
    if not trace_id.isprintable():
        trace_id = uuid.uuid4().hex[:16]
    with request_context.bind(trace_id=trace_id):
        response = await call_next(request)
    response.headers["X-Request-ID"] = trace_id
    return response
```

The inbound header is **untrusted input**: it is length-capped, and rejected if it contains non-printable characters. Without that, a client could send a header containing a newline and forge extra log lines (log injection).

2. `current_user` (the auth dependency) calls `request_context.set_user(user.github_user_id)` after the session is validated.

This is for correlation only. **Authorization still comes from the session object**, never from the ContextVar.

3.5 Making logs use it

`install_logging()` installs a `LogRecord` factory that adds `trace_id` and `user_id` to every record:

```
2026-… INFO resolveflow.execute trace=9f2c… user=1234567 github write repo=acme/widgets issue=7
label=bug
```

A record factory (rather than a handler filter) works for any handler: uvicorn's, pytest's, whatever is added later.

`graph/nodes/execute.py`, the only node with side effects, now logs a write audit line. That line runs in a LangGraph **worker thread**, and still carries the right ids because the context is copied into it.

3.6 Propagation rules (the part interviewers probe)

Mechanism	Does the context come along?
`await` in the same task	Yes (same context)
`asyncio.create_task(...)`	Yes (copied at creation)
`asyncio.to_thread(...)`	**Yes** (explicitly copied)
LangGraph sync node under `ainvoke`	**Yes**, verified by a test, not assumed
Plain `threading.Thread(...)`	**No**, it starts empty. Use `contextvars.copy_context().run(fn)`
Values set *inside* a child task	Not visible to the parent (copies are independent)

All of these are covered by tests, including the negative one (the bare thread), so the caveat is documented by an executable example.

3.7 Why this connects to Karan's background

His own writing on observability emphasises propagating `traceId`, `tenantId` and `userId` as correlation attributes across every hop of a distributed system. Per-request context that survives thread hops, plus an identifier returned to the client so a support ticket can quote it, is the in-process version of that idea. Say it as: "this is the in-process half of correlation propagation; across services it would be W3C `traceparent` headers or OpenTelemetry context."

3.8 Follow-ups to be ready for

- "*Why not a global dict keyed by request id?*" You would have to pass the id everywhere to look it up, and you must clean it up manually. ContextVar does both automatically.
- "*ContextVar vs thread-local?*" Thread-locals are per thread, but many asyncio tasks share one thread, so thread-locals would leak between concurrent requests. ContextVars are per task.
- "*Is this safe for security decisions?*" Deliberately not used for them. It is observability only.

Chapter 4. Semaphore: bounding `/api/analyze`

4.1 The problem

Each analyze run does paid and rate-limited work: OpenAI calls, Pinecone queries, GitHub reads, running graph nodes in worker threads. With no cap, a burst of N requests means N concurrent LLM calls (provider 429s, cost spikes) and N occupied threads (starving everything else, including `/api/health`, which Render uses to decide whether the instance is alive).

There is already a per-user **daily** quota (`db.increment_and_check_daily_limit`), but that bounds *volume per day*, not *concurrency right now*. Those are different controls.

4.2 What a semaphore is

An `asyncio.Semaphore(n)` is a counter of available "slots". `acquire()` takes one, waiting if none are free; `release()` gives it back. With `n = 4`, at most four holders exist at once.

4.3 The code (`app/limits.py`)

```
class AnalysisGate:
    def __init__(self, limit, wait_seconds):
        if limit < 1:
            raise ValueError("limit must be >= 1")
        self._sem = asyncio.Semaphore(limit)
        self._wait_seconds = wait_seconds

    @asynccontextmanager
    async def slot(self):
        try:
            await asyncio.wait_for(self._sem.acquire(), timeout=self._wait_seconds)
        except asyncio.TimeoutError:
            raise GateBusy from None
        try:
            yield
        finally:
            self._sem.release()
```

Design points:

- **Bounded wait, then shed load.** After `ANALYZE_QUEUE_WAIT_SECONDS` (default 10) the caller gets `GateBusy`, which `main.py` turns into **503 + `Retry-After: 10`**. The alternative, queueing forever, holds client connections open and hides overload until a proxy times out.
- **Release only in `finally`.** If the body raises, or the task is cancelled (the client disconnected), the slot is still released. A leaked slot is the classic semaphore bug: capacity shrinks silently until the service wedges. Two tests cover exception and cancellation.
- **Acquire is outside the `try`.** If acquisition fails or times out, we never release a slot we do not hold. (Releasing a semaphore you did not acquire inflates capacity, the opposite bug.)
- **Context manager API.** Callers cannot forget to release: `async with gate.slot(): ...`.

4.4 Where it is used (`app/main.py`)

```
try:
    async with _state["analysis_gate"].slot():
        if not await db.increment_and_check_daily_limit(...):   # quota check
            raise HTTPException(429, ...)
        ...
        result = await _state["compiled_graph"].ainvoke(...)
        await db.record_thread_owner(...)
except GateBusy:
    raise HTTPException(503, "...retry shortly", headers={"Retry-After": "10"}) from None
```

4.5 Subtle decisions

Decision	Reason
Slot is taken **before** the daily-quota increment	If the request is shed with a 503, the user must not lose one of their daily analyses for work that never ran.
Gate created in `lifespan`, not at import	An `asyncio.Semaphore` is associated with an event loop. Creating it at import time can bind it to the wrong loop (or none). Building it inside the running app avoids that.
`/api/resume` is **not** gated	A human already approved. The remaining work is light, and shedding it would strand an approved action.
Limit is configurable by env var	`MAX_CONCURRENT_ANALYSES` (default 4), `ANALYZE_QUEUE_WAIT_SECONDS` (default 10).
Per process, not global	`render.yaml` allows `maxInstances: 3`, so the true ceiling is `limit × instances` (12). A global limit needs shared state such as Redis or Postgres, which would be a different, larger change.

4.6 Semaphore vs the neighbours

Control	Bounds	Here
Semaphore (this)	concurrent in-flight work	analyze runs per process
Daily quota (existing)	volume per user per day	`DAILY_ANALYZE_LIMIT = 30`
Rate limiter (token bucket)	requests per unit time	not implemented
Thread pool size	worker threads	indirectly bounded by the gate

A semaphore says "no more than N at once". It does not say "no more than N per second". Be precise about that if asked.

4.7 Follow-ups to be ready for

- "*Semaphore vs Lock?*" A Lock is a semaphore with one slot (mutual exclusion). A semaphore permits N.
- "*Semaphore vs BoundedSemaphore?*" `BoundedSemaphore` raises if you release more than you acquired, catching the over-release bug. Here the `finally` structure already makes that impossible.

- "*Why 503 and not 429?*" 429 means *you* sent too many requests. 503 means *the server* is temporarily unable to handle the request, which is what a full gate is.
- "*What happens to waiters when one is cancelled?*" `asyncio.Semaphore` handles cancelled waiters correctly and wakes the next one.
- "*What about fairness?*" `asyncio.Semaphore` wakes waiters in FIFO order.

Chapter 5. The GIL: what we did and did not do

5.1 You cannot "implement" the GIL

It is a property of the CPython interpreter, not a feature you add. What we did is **make correct decisions given the GIL**, and document them in `docs/CONCURRENCY.md`.

5.2 The decisions

Question	Answer in this codebase
Does the GIL limit the request path?	No. Everything slow is network-bound (GitHub, OpenAI, Pinecone, Postgres); waiting releases the GIL.
Do we need `multiprocessing`?	No. There is no CPU-heavy pure-Python hot path in the request flow.
Does the GIL make our shared state safe?	**No**, and we do not rely on it. The only process-wide dict is `_state` in `main.py`, written once during `lifespan`. Per-user state lives in Postgres.
Where would the GIL matter?	CPU-bound pure-Python work such as chunking a very large corpus in `ingest.py`. That is a one-shot script, so we left it alone.

5.3 The sentence to remember

> "The GIL protects the interpreter, not my logic. It doesn't slow down code that is waiting on the network, and it doesn't make `x += 1` atomic."

(In the earlier deep dive we demonstrated this: four threads incrementing a counter lost updates, 3,227,333 instead of 4,000,000, on Python 3.9.)

Chapter 6. Tests: what each one proves

`tests/test_concurrency.py`, 18 tests, no network and no API keys. GitHub is replaced with fakes that `time.sleep(0.15)`, which blocks exactly like `requests` does.

Test	Proves
`test_async_fetch_matches_sync_result`	Async path returns an identical dict to the sync path
`test_async_fetch_overlaps_independent_calls`	Wall time is about the longest chain, not the sum
`test_async_fetch_runs_blocking_calls_on_worker_threads`	Blocking calls do not run on the event-loop thread
`test_check_runs_only_fetched_after_linked_pr`	The dependency is respected; no PR means no check-runs call
`test_async_fetch_propagates_original_exception_type`	Errors keep their type (`requests.HTTPError`)
`test_bad_url_still_raises_value_error`	Input validation unchanged
`test_context_reaches_to_thread_workers`	Context is copied into `to_thread`
`test_concurrent_tasks_do_not_see_each_others_context`	Isolation between interleaved requests
`test_bind_restores_previous_value_even_on_error`	Token reset on exceptions, nesting works
`test_plain_threading_thread_does_not_inherit_context`	The documented caveat (negative test)
`test_langgraph_sync_node_sees_request_context`	Context reaches sync LangGraph nodes under `ainvoke`
`test_log_records_carry_trace_and_user`	Log records are stamped; defaults outside a request
`test_gate_never_exceeds_limit`	Peak in-flight is exactly the limit with 10 contenders
`test_gate_sheds_load_after_wait_budget`	`GateBusy` after the wait budget
`test_gate_releases_slot_when_body_raises`	No slot leak on exceptions
`test_gate_releases_slot_when_task_is_cancelled`	No slot leak on client-disconnect cancellation
`test_gate_rejects_nonpositive_limit`	Config validation
`test_graph_runs_async_and_sync_paths_to_same_evidence`	Both `invoke` and `ainvoke` work through the real compiled graph

Mutation check (we broke the code on purpose to confirm the tests can fail): - Making the fetch sequential made `test_async_fetch_overlaps_independent_calls` fail. - Raising the semaphore limit made `test_gate_never_exceeds_limit` fail (peak 7 instead of 2) and `test_gate_sheds_load_after_wait_budget` fail.

A test that cannot fail proves nothing. This is worth saying in an interview: "I checked the tests fail when the behaviour is broken."

Run: `uv run pytest tests/test_concurrency.py`

Chapter 7. Honesty table: what you can and cannot claim

Claim	Status	Say it as
"I parallelised independent GitHub reads with asyncio"	**True, in code, tested**	State the dependency structure (the PR→checks chain).
"It reduced latency by X%"	**Not measured on the real system**	"In a simulated test with 150 ms calls it went from 0.77 s to 0.31 s. I'd measure live before quoting a number."
"Trace/user IDs are propagated into logs and worker threads"	**True, tested**	Mention the bare-thread caveat; it shows depth.
"I rate-limit the API"	**Imprecise**	It is a *concurrency cap* (semaphore), not a rate limiter. The daily quota is separate.
"It scales horizontally"	**Careful**	The cap is per process; with up to 3 Render instances the ceiling is 12.
"I removed the GIL bottleneck"	**False**	The GIL was never the bottleneck.
"It is deployed and running in production"	**Not yet**	It is on a branch, tested with fakes, **not smoke-tested against live Postgres/OpenAI/Pinecone/GitHub**, not merged.
"Github token handling is secure"	**Known gap**	See below.

Known issue found during this work (not fixed)

`github_token` is part of `GraphState` and is passed as input to `ainvoke`, so LangGraph's Postgres checkpointer likely persists each user's GitHub OAuth token in the checkpoint tables, in addition to `sessions.github_token`. This was read from the code, not confirmed in a live database. Suggested fix: keep the token out of graph state (look it up from the session inside `execute`, or pass it via `config["configurable"]`), and encrypt or expire tokens at rest. If Karan asks "what would you fix next?", this is an excellent, honest answer because it shows you audit your own security posture.

Chapter 8. Talk track and drill

The three-minute walkthrough (structure)

1. **Context (15 s).** "ResolveFlow runs a LangGraph pipeline behind FastAPI. All the latency is network I/O."
2. **Problem 1 (45 s).** Sequential fetch → dependency graph → `gather` over `to_thread` → why not `httpx`, why not `TaskGroup`.
3. **Problem 2 (45 s).** Request identity → ContextVar → propagation rules → used for audit on the GitHub write.
4. **Problem 3 (45 s).** Overload → semaphore with bounded wait → 503 → slot taken before quota → per-process caveat.
5. **GIL (15 s).** "Not the bottleneck here; doesn't protect my logic."
6. **Verification and limits (15 s).** 18 tests with mutation checks; not yet measured live; token-in-checkpoint finding.

Drill (answer aloud, cover the chapters above)

1. Why can't you simply `gather` all five GitHub calls?
2. Why does `to_thread` give real parallelism for `requests` when there is a GIL?
3. Why `gather` and not `TaskGroup` here? Give two reasons.
4. A global variable would hold the trace id. What breaks, concretely, with two concurrent requests?
5. Which of these propagate a ContextVar: `await`, `create_task`, `to_thread`, `threading.Thread`?
6. Why is the inbound `X-Request-ID` header length-capped and checked for printable characters?
7. Your semaphore has limit 4 and 10 requests arrive together. Walk through what each request experiences.
8. Why is the slot acquired before the daily-quota increment?
9. What bug appears if you `release()` in a `finally` but the `acquire()` was inside the `try`?
10. Why is the semaphore built in `lifespan` rather than at module import?
11. Semaphore vs rate limiter: what does each guarantee?
12. With `maxInstances: 3`, what is the real concurrency ceiling, and how would you make it global?
13. What did the mutation check prove?
14. What is the first thing you would do before putting this in front of real traffic?

Model answers (short)

1. `get_check_runs` needs the head SHA from `get_linked_pull_request`, so those two are a chain; only the independent reads can start together.
2. A thread blocked on a socket has released the GIL, so several threads can all be waiting on the network at once. The GIL only serialises running bytecode.
3. A thread already blocked in `requests` cannot be cancelled, so TaskGroup's cancel-siblings would not stop them; and TaskGroup wraps failures in `ExceptionGroup`, degrading the 502 message. The calls are read-only so stragglers are harmless.
4. Request B overwrites the value while A is mid-flight, so A's logs (and the GitHub write audit line) get B's identity: wrong attribution and a privacy leak.
5. `await` yes, `create_task` yes (copied), `to_thread` yes (copied), bare `threading.Thread` no.

6. It is untrusted input. Without a cap, large values bloat logs; with newlines, a client could forge log lines.
7. Four start immediately. Six wait up to 10 s for a slot. As runs finish, waiters get slots in FIFO order. Any still waiting at 10 s get 503 with `Retry-After`, and those requests never consumed daily quota.
8. A shed request did no work, so it must not cost the user a daily analysis.
9. If acquire fails or times out you would release a slot you never held, inflating capacity beyond the limit.
10. A semaphore binds to an event loop; creating it inside the running app avoids binding to the wrong loop.
11. A semaphore bounds how many run at once. A rate limiter bounds how many start per unit time. They answer different questions.
12. 12 (4 × 3). A global cap needs shared state, e.g. a Redis counter or Postgres advisory lock/lease with expiry.
13. That the tests detect the behaviour they claim to: breaking the code made them fail.
14. Smoke-test against live services, measure real before/after latency, then address the token-in-checkpoint issue.

Chapter 9. try / except / finally in async code

Every example here was run on Python 3.12 and the outputs shown are real. This chapter explains *why* `app/limits.py` is written the way it is, and it is the most likely place for a follow-up probe.

9.1 The basic shape

```
async def fetch():
    try:
        await asyncio.sleep(0.01)
        raise ValueError("boom")
    except ValueError as e:
        print("except:", e)
    else:
        print("else")                # only if no exception
    finally:
        await asyncio.sleep(0)       # awaiting inside finally is allowed
        print("finally (awaited ok)")
```

Output: `except: boom`, then `finally (awaited ok)`. The structure is the same as sync Python, but **`await` is legal inside `except` and `finally`**. That is what lets you close connections or release locks asynchronously.

9.2 Cancellation: the part interviewers care about

```
async def worker():
    try:
        await asyncio.sleep(10)
    except asyncio.CancelledError:
        print("cleanup on cancel")
        raise                        # MUST re-raise
    finally:
        print("finally runs on cancel")

t = asyncio.create_task(worker())
await asyncio.sleep(0.05)
t.cancel()
await t                              # the caller sees CancelledError
```

Output:

```
cleanup on cancel
finally runs on cancel
caller saw CancelledError
```

`task.cancel()` does not kill the task. It schedules `CancelledError` to be raised **at the task's next `await`**, so `finally` blocks run normally.

Bug: swallowing it

```
except asyncio.CancelledError:
    print("swallowed")               # no re-raise
```

Output: `swallowed`, and then the task **finished normally** (`t.cancelled()` was `False`). The caller believes the work completed. Timeouts and `TaskGroup` depend on cancellation propagating, so swallowing it breaks them.

`except Exception` does not catch it. Since Python 3.8, `CancelledError` derives from `BaseException`, not `Exception`. In the run, an `except Exception` branch never printed while `finally` still ran. This is correct behaviour, and it is why a bare `except:` or `except BaseException:` is dangerous in async code.

9.3 Timeouts are cancellation too

```
try:
    async with asyncio.timeout(0.05):
        try:
            await asyncio.sleep(1)
        finally:
            print("finally on timeout")
except TimeoutError:
    print("TimeoutError")
```

Output: `finally on timeout`, then `TimeoutError`. `timeout` cancels the body; when the resulting `CancelledError` exits the block, it is converted into `TimeoutError`. That is why swallowing `CancelledError` in the middle breaks timeouts.

9.4 Cleanup can itself be cancelled

```
async def worker():
    try:
        await asyncio.sleep(10)
    finally:
        await asyncio.sleep(0.2)     # slow cleanup, e.g. closing a connection
        print("cleanup finished")    # never printed if cancelled a second time
```

Cancelled twice, 50 ms apart, the output was `cleanup itself cancelled`: the second cancel interrupted the cleanup, so the connection would never have closed.

Fix: shield cleanup that must finish

```
finally:
    await asyncio.shield(close_connection())
```

Output: `shielded cleanup done`, while the caller still saw `CancelledError`. Caveat: `shield` protects the inner task, but the outer `await` can still be cancelled, and the inner work then finishes in the background. Use it sparingly, only for must-finish cleanup.

9.5 `gather`, `return_exceptions` and `TaskGroup`

```
# default gather: first exception propagates, the OTHERS KEEP RUNNING
try:
    await asyncio.gather(bad(), slow())
except KeyError: ...                # "slow finished anyway" prints afterwards

# collect everything instead of raising
r = await asyncio.gather(bad(), ok(), return_exceptions=True)
# [KeyError('x'), 'ok']

# TaskGroup (3.11+): first failure cancels siblings; errors arrive as an ExceptionGroup
try:
    async with asyncio.TaskGroup() as tg:
        tg.create_task(bad())
        tg.create_task(asyncio.sleep(1))
except* KeyError as eg:
    print(eg.exceptions)            # (KeyError('x'),)
```

This is why `afetch_evidence` uses `gather`: a thread already blocked in `requests` cannot be cancelled, and `TaskGroup` would wrap the original error in an `ExceptionGroup`.

9.6 The real example in ResolveFlow

`AnalysisGate.slot()` in `app/limits.py`:

```
await asyncio.wait_for(self._sem.acquire(), timeout=self._wait_seconds)   # timeout cancels acquire
try:
    yield
finally:
    self._sem.release()    # runs on success, on exception AND on client-disconnect cancellation
```

Two deliberate choices: - `acquire()` is **outside** the `try`. If it times out, we never reach `finally`, so we never release a slot we do not hold (which would inflate capacity above the limit). - `release()` is **only** in `finally`, so no path (exception, cancellation) can leak a slot.

`test_gate_releases_slot_when_task_is_cancelled` and `test_gate_releases_slot_when_body_raises` prove both.

9.7 Probes and answers

Probe	Answer
"What does `task.cancel()` do?"	Schedules `CancelledError` to be raised at the task's next `await`. It is not instant, and the task can still run `finally`.
"Why is `CancelledError` a `BaseException`?"	So `except Exception` does not accidentally swallow cancellation.
"What if I catch it and don't re-raise?"	The task looks like it completed normally, and timeouts and TaskGroups break. Always re-raise.
"Can you `await` in `finally`?"	Yes, but that await can be cancelled again. Use `shield` for must-finish cleanup.
"How does `asyncio.timeout` work?"	It cancels the body, then converts the resulting `CancelledError` into `TimeoutError` at the block boundary.

Probe	Answer
"`gather` vs `TaskGroup` on failure?"	`gather` re-raises the first error and leaves the others running. `TaskGroup` cancels siblings and raises an `ExceptionGroup`, handled with `except*`.
"What does `return_exceptions=True` do?"	Returns exceptions as ordinary results instead of raising, so you must inspect each one.
"A client disconnects mid-request?"	The server cancels the handler task. Without `try/finally` you leak locks, semaphore slots and connections.
"Can you cancel a thread started by `to_thread`?"	No. The await is cancelled, but the thread runs to completion.
"Where should `acquire` go relative to `try`?"	Outside it, with `release` in `finally`; otherwise a failed acquire releases a slot you never held.

9.8 One-liners to memorise

- Cleanup goes in `finally`.
- Catch `CancelledError` only to clean up, and **always re-raise**.
- Never use a bare `except:`.
- Acquire outside the `try`, release inside `finally`.
- Use `shield` only for cleanup that must finish.

9.9 Drill

1. A task is cancelled while awaiting an HTTP call. In what order do `except CancelledError`, `finally`, and the caller's `await` resolve?
2. Why does `except Exception:` not catch cancellation, and why does that matter?
3. You wrapped a call in `try/except Exception: log and continue` inside a retry loop. What goes wrong when the request is cancelled?
4. How would you make sure a database connection is closed even if the request is cancelled twice?
5. Explain exactly how `asyncio.timeout` produces a `TimeoutError`.
6. Why is `acquire()` outside the `try` in `AnalysisGate.slot()`?
7. `gather` raised `KeyError` but a sibling is still running. Is that a bug, and what would you use instead?
8. You called `await asyncio.to_thread(...)` and the client disconnected. Does the work stop?

Model answers: (1) `CancelledError` is raised in the task at its await; the `except CancelledError` handler runs, then `finally`, then the caller's `await task` raises `CancelledError`. (2) It derives from `BaseException`; otherwise broad handlers would swallow cancellation and tasks would never stop. (3) A retry loop that catches broad exceptions must let `CancelledError` through (it already does with `except Exception`, but a bare `except:` would retry a cancelled request forever). (4) Put the close in `finally` and wrap it in `asyncio.shield`. (5) It cancels the body, and at the block exit converts that `CancelledError` into `TimeoutError`. (6) So a timed-out acquire never releases a slot it did not take. (7) Not a bug; `gather` does not cancel siblings. Use `TaskGroup` if you want fail-fast cancellation and your work is actually cancellable. (8) No; the thread continues, only the await is abandoned.

Appendix. Files changed

File	Change
`graph/nodes/fetch_evidence.py`	+ `_parse_issue_url`, + `afetch_evidence` (sync `fetch_evidence` kept)
`graph/build.py`	`RunnableLambda(fetch_evidence, afunc=afetch_evidence)`
`graph/request_context.py`	**new**: ContextVars, `bind`, `set_user`, `install_logging`
`app/limits.py`	**new**: `AnalysisGate`, `GateBusy`
`app/main.py`	gate in lifespan; tracing middleware; `set_user` in `current_user`; `/api/analyze` wrapped in the gate with 503 handling; log lines
`graph/nodes/execute.py`	audit log line before GitHub writes
`tests/test_concurrency.py`	**new**: 18 tests
`docs/CONCURRENCY.md`, `CHANGELOG.md`	rationale and changelog entry